

머신러닝

비지도 학습

Chapter 12



Prof. Yang



한국성서대학교
KOREAN BIBLE UNIVERSITY

목 차

CONTENTS

01 K-평균 개념

02 K-평균 한계점 해결

03 평가 방식: 실루엣 그래프

04 실제 데이터로 사이킷런 실습

01

K-평균 개념



k-평균(k-means)?

- k-평균은 산업 현장은 물론 학계에서도 널리 사용
- 군집은 비슷한 객체로 이루어진 그룹을 찾는 기법
- 한 그룹 안의 객체들은 다른 그룹에 있는 객체보다 더 관련되어 있음
- 군집에 대한 상용 애플리케이션의 예로는 문서나 음악, 영화를 여러 주제의 그룹으로 모으는 경우를 들 수 있음
- 또는 추천 엔진에서 하듯이 구매 이력의 공통 부분을 기반으로 관심사가 비슷한 고객을 찾는 것
- k-평균 알고리즘은 구현하기 매우 쉽고 다른 군집 알고리즘에 비해 계산 효율성이 높기 때문에 인기가 많음
- k-평균 알고리즘이 원형 클러스터를 구분하는 데 뛰어나지만 이 알고리즘 단점은 사전에 클러스터 개수 k 를 지정해야 하는 것
- 적절하지 않은 k 를 고르면 군집 성능이 좋지 않음

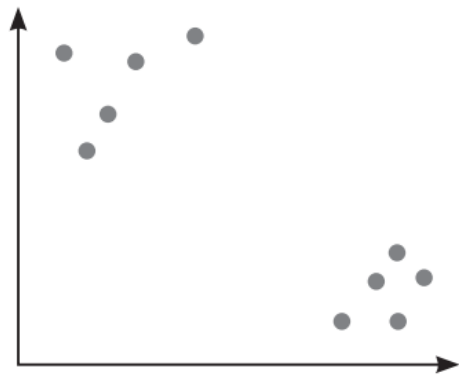


k-평균(k-means) 알고리즘 과정

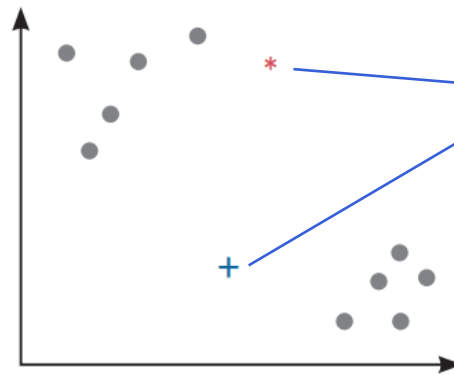
- 실제 군집 애플리케이션에서는 샘플에 대한 진짜 카테고리 정보(추론이 아니라 실증적인 정보)가 전혀 없음
 - 클래스 레이블이 있다면 이 작업은 지도 학습에 해당
- 클러스터링의 목표는 특성의 유사도에 기초하여 샘플을 그룹으로 모으는 것
- k-평균 알고리즘 과정
 1. 샘플 포인트에서 랜덤하게 k개의 센트로이드를 초기 클러스터 중심으로 선택
 2. 각 샘플을 가장 가까운 센트로이드 $\mu(j), j \in \{1, \dots, k\}$ 에 할당
 3. 할당된 샘플들의 중심으로 센트로이드를 이동
 4. 클러스터 할당이 변하지 않거나 사용자가 지정한 허용 오차나 최대 반복 횟수에 도달할 때 까지 단계 2와 3을 반복



k-평균(k-means) 알고리즘 과정

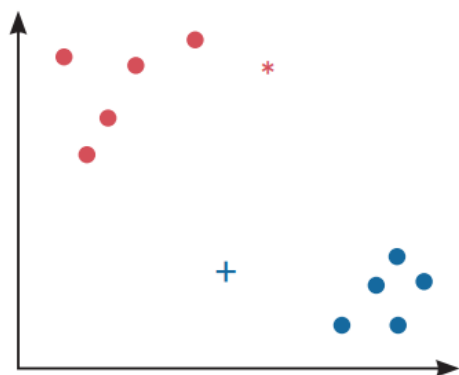


Step. 1

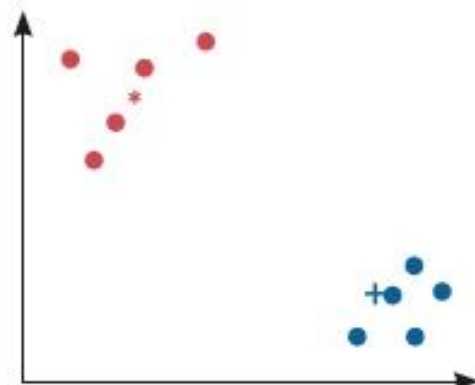


Step. 2

센트로이드



Step. 3

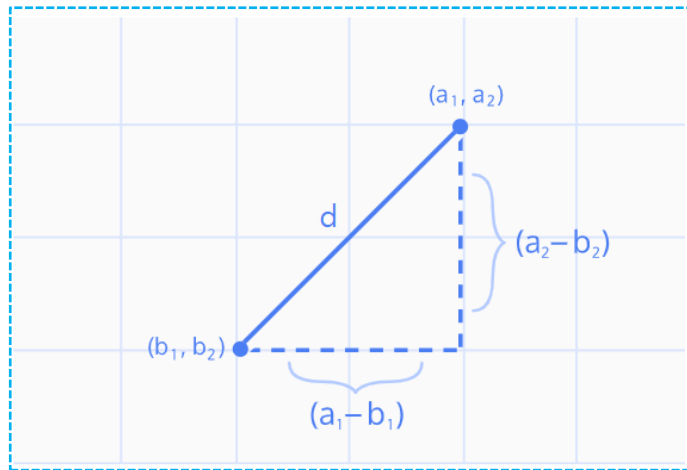


Step. 4



유사도 정의

- 연속적인 특성을 가진 샘플을 클러스터로 묶는 데 널리 사용되는 거리는 m-차원 공간에 있는 두 포인트 x와 y 사이의 유클리디안 거리의 제곱(squared Euclidean distance)



$$\sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \dots + (p_n - q_n)^2} = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

$$d(x, y)^2 = \sum_{j=1}^m (x_j - y_j)^2 = \|x - y\|_2^2$$

- j: 샘플 포인트 x와 y의 j번째 열



오차 및 최적화

- 유클리디안 거리 지표를 기반으로 간단한 최적화 문제로 k-평균 알고리즘을 기술할 수 있음
- 클러스터 내 제곱 오차합(SSE, Sum of Squared Error) 또는 클러스터 관성(cluster inertia)을 반복적으로 최소화하는 방법

$$SSE = \sum_{i=1}^n \sum_{j=1}^k w^{(i,j)} \left\| \mathbf{x}^{(i)} - \boldsymbol{\mu}^{(j)} \right\|_2^2$$

i: 각각 샘플(데이터 레코드) 인덱스
j: 클러스터 인덱스

- 여기서 $\mu(j)$ 는 클러스터 j의 대표 포인트(센트로이드)
- 샘플 $x(i)$ 가 클러스터 j 안에 있다면 $w(i, j) = 1$ 이고, 아니면 $w(i, j) = 0$

$$w^{(i,j)} = \begin{cases} 1 & x^{(i)} \in j \text{ 이면} \\ 0 & \text{그렇지 않으면} \end{cases}$$



사이킷런 실습-데이터 준비

임의의 가짜 데이터 준비

```
import numpy as np

center = np.random.uniform(0,100,3)#3개의 센터

X0 = np.random.normal(center[0],10,100).reshape(-1,2)
X1 = np.random.normal(center[1],10,100).reshape(-1,2)
X2 = np.random.normal(center[2],10,100).reshape(-1,2)

X = np.vstack([X0, X1, X2])#합치기

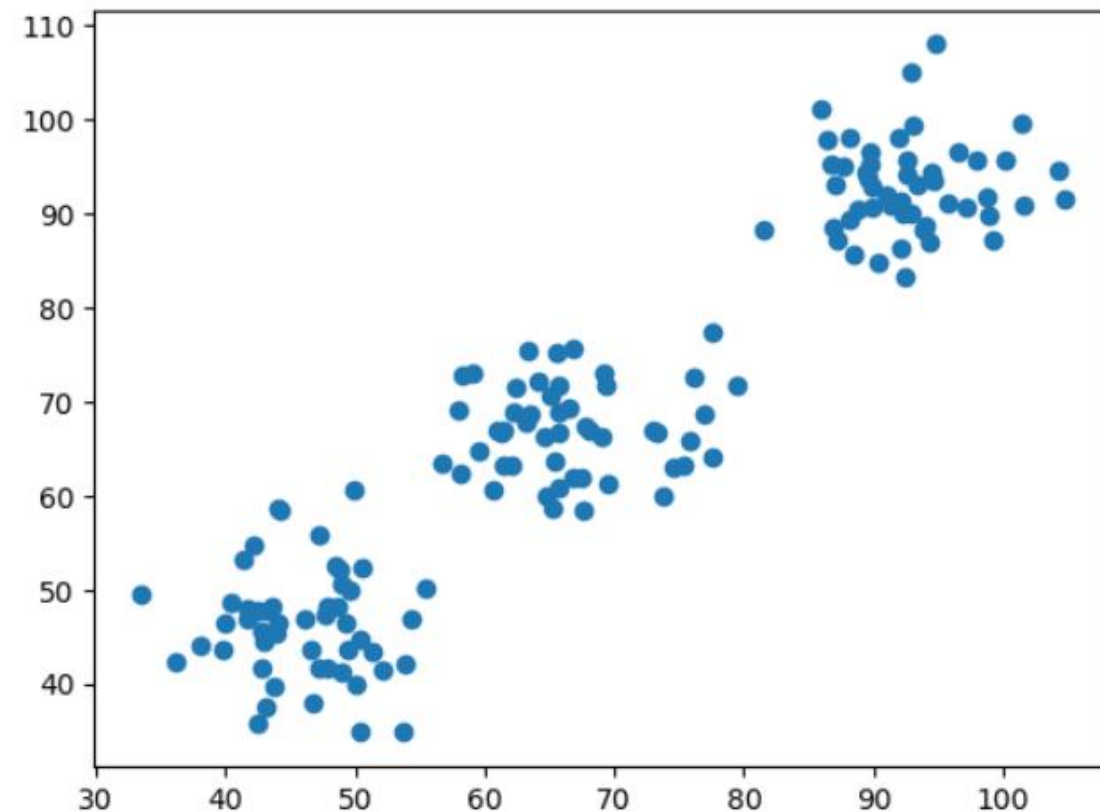
#랜덤하게 섞기
idx = np.random.choice(150, size=150, replace=False)
X = X[idx,:]

X[:10]
```

```
array([[ 87.19667437,  87.21969652],
       [ 94.75420366, 107.99591411],
       [ 88.1217466 ,  89.34756353],
       [ 42.43389378,  35.68638417],
       [ 44.15320818,  58.5251678 ],
       [ 49.96205471,  60.57453655],
       [ 90.30319417,  84.82680282],
       [ 73.73928747,  60.02775302],
       [ 53.76063928,  35.00228084],
       [ 65.67542399,  60.91724771]])
```

```
import matplotlib.pyplot as plt
```

```
plt.scatter(X[:,0], X[:,1])
plt.show()
```





사이킷런 실습-학습하기

- cluster 모듈의 KMeans 클래스를 적용

```
from sklearn.cluster import KMeans
```

```
km = KMeans(n_clusters=3,  
            init='random',  
            n_init=10,  
            max_iter=300,  
            tol=1e-04,  
            random_state=0)
```

```
y_km = km.fit_predict(X)
```

```
y_km#각 데이터 별 cluster number
```

```
array([0, 0, 0, 1, 1, 1, 0, 2, 1, 2, 2, 1, 2, 1, 1, 1, 0, 0, 1, 1, 0, 0,  
       1, 0, 0, 1, 2, 2, 0, 1, 2, 0, 0, 2, 2, 2, 2, 2, 0, 1, 2, 1, 0, 1,  
       2, 2, 0, 1, 2, 1, 1, 1, 0, 1, 2, 1, 0, 0, 2, 1, 0, 0, 2, 1, 1, 1,  
       0, 2, 2, 0, 1, 2, 0, 2, 0, 0, 1, 1, 0, 1, 0, 0, 2, 0, 2, 2, 0, 2,  
       2, 2, 2, 2, 0, 1, 2, 0, 0, 0, 2, 2, 2, 0, 1, 2, 1, 1, 0, 1, 2, 1,  
       2, 0, 2, 1, 1, 1, 1, 2, 1, 0, 0, 1, 2, 0, 0, 2, 1, 0, 0, 1, 0, 2,  
       0, 0, 0, 2, 1, 1, 1, 2, 0, 2, 1, 2, 2, 1, 2, 2, 0, 1])
```

p.9 X데이터와 클러스터 결과

```
array([[ 87.19667437,  87.21969652],→0  
       [ 94.75420366, 107.99591411],→0  
       [ 88.1217466 ,  89.34756353],→0  
       [ 42.43389378,  35.68638417],→1  
       [ 44.15320818,  58.5251678 ],→1  
       [ 49.96205471,  60.57453655],→1  
       [ 90.30319417,  84.82680282],→0  
       [ 73.73928747,  60.02775302],→2  
       [ 53.76063928,  35.00228084],→1  
       [ 65.67542399,  60.91724771]]→2
```



KMeans 매개변수

- `n_clusters`: 클러스터 개수를 3으로 지정했음
 - 클러스터 개수를 사전에 지정해야 하는 것은 k-평균 알고리즘의 한계 중 하나
- `n_init = 10`으로 설정하면 k-평균 군집 알고리즘을 각기 다른 랜덤한 센트로이드에서 독립적으로 열 번 실행하여 가장 낮은 SSE를 만드는 하나를 최종 모델로 선택
- `max_iter`: 한 번의 실행에서 수행할 최대 반복 횟수를 지정함
 - 여기서는 300
 - 사이킷런의 k-평균 구현은 최대 반복 횟수에 도달하기 전에 수렴하면 일찍 종료
- `tol`: 수렴을 결정하는 클러스터 내 제곱 오차합의 변화량에 대한 허용 오차를 조정함
 - 앞 코드에서는 허용 오차를 $1e-04 (=0.0001)$ 로 선택
- `random_state`: 랜덤 시드로써 결과 재현을 위해 고정



사이킷런 실습-학습 결과

- 센트로이드 좌표는 "모델명.cluster_centers_"

```
km.cluster_centers_
```

```
array([[92.69299031, 92.84441807],  
       [45.99442557, 46.27083902],  
       [66.62657764, 67.0763369 ]])
```

Cluster number	x_1 센트로이드 좌표	x_2 센트로이드 좌표
0	92.69299031	92.84441807
1	45.99442557	46.27083902
2	66.62657764	67.0763369

- 센트로이드의 좌표는 각 클러스터에 속한 데이터들의 평균값으로 결정됨

```
for i in range(3):  
    print(np.mean(X[y_km==i,:],axis=0))
```

```
array([[92.69299031, 92.84441807],  
       [45.99442557, 46.27083902],  
       [66.62657764, 67.0763369 ]])
```



사이킷런 실습-학습 결과

- 센트로이드 좌표는 "모델명.cluster_centers_"

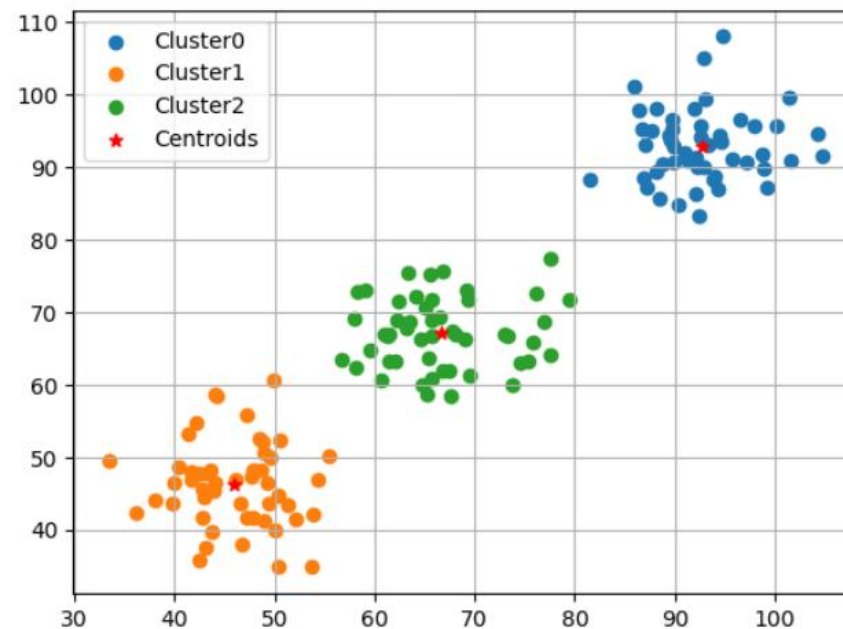
```
km.cluster_centers_
```

```
array([[92.69299031, 92.84441807],  
       [45.99442557, 46.27083902],  
       [66.62657764, 67.0763369 ]])
```

Cluster number	x_1 센트로이드 좌표	x_2 센트로이드 좌표
0	92.69299031	92.84441807
1	45.99442557	46.27083902
2	66.62657764	67.0763369

- 시각화

```
cluster = np.unique(y_km) #클러스터 넘버  
for n in cluster:  
    plt.scatter(X[y_km == n, 0],  
                X[y_km == n, 1],  
                label='Cluster'+str(n))  
  
plt.scatter(km.cluster_centers[:, 0],  
            km.cluster_centers[:, 1],  
            marker='*', c='red', label='Centroids')  
  
plt.legend()  
plt.grid()  
plt.show()
```



02

K-평균 한계점 해결



K-평균 한계점

1. 클러스터의 개수를 몇으로 선택할지는 실제 애플리케이션에서는 명확하지 않을 수 있음
 - 특히 시각화할 수 없는 고차원 데이터셋에서 그러함
2. 클러스터가 중첩되지 않고 계층적이지 않다는 것
3. 빈 클러스터가 발생할 수 있음



1) k-평균 ++(k-means++)

- 언급한 k-평균의 가정과 단점을 해결하지는 못하지만 초기 클러스터 중심을 똑똑하게 할당하여 군집 결과의 품질을 크게 향상시킴
 - 초기 센트로이드를 랜덤하게 할당하는 기본 k-평균 알고리즘을 다루었음
 - 초기 센트로이드가 좋지 않게 선택되면 이따금 나쁜 군집 결과를 만들거나 수렴이 느려짐
 - 이 문제를 해결하는 한 가지 방법은 같은 데이터셋에서 k-평균 알고리즘을 여러 번 실행하여 SSE (Sum of Squared Errors) 입장에서 가장 성능이 좋은 모델을 선택하는 것
 - 또 다른 방법은 k-평균++ 알고리즘을 통해 초기 센트로이드가 서로 멀리 떨어지도록 위치시키는 것
 - 이는 기본 k-평균보다 일관되고 훌륭한 결과를 만듦

코드:

```
kmp = KMeans(n_clusters=3,  
             init='k-means++',  
             n_init=10,  
             max_iter=300,  
             tol=1e-04,  
             random_state=0)  
kmp.fit(X) #학습
```



```
KMeans  
KMeans(n_clusters=3, n_init=10, random_state=0)
```




2) 엘보우 방법을 사용하여 최적의 클러스터 개수 찾기

- 비지도 학습에서 가장 어려운 점 하나는 최종 답을 모른다는 것
- 데이터셋에 진짜 클래스 레이블이 없기 때문에 지도 학습의 성능 평가를 위해 사용한 기법들을 적용할 수 없음
- 군집 품질을 평가하려면 알고리즘 자체의 지표를 사용해야 함
- 예를 들어 k-평균 군집의 성능을 비교하기 위해 클래스 내 SSE(Sum of Squared Errors) 를 사용

$$SSE = \sum_{i=1}^k \sum_{x \in C_i} ||x - \underline{\mu_i}||^2$$

평균=센트로이드

- 다행히 사이킷런을 사용하면 클래스 내 SSE를 직접 계산할 필요가 없음
- KMeans 모델을 학습하면 inertia_ 속성에 이미 계산되어 있음

```
print('Base SSE: %.2f' % km.inertia_)  
print('k-mean++ SSE: %.2f' % kmp.inertia_)
```



```
Base SSE: 6866.82  
k-mean++ SSE: 6866.82
```



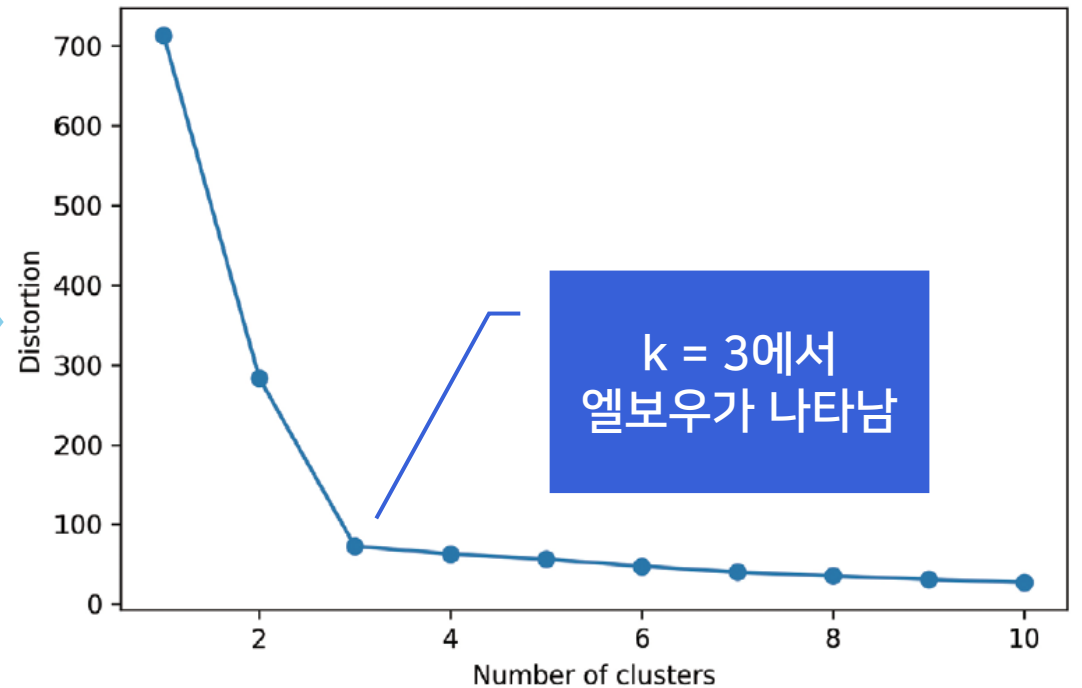
2) 엘보우 방법을 사용하여 최적의 클러스터 개수 찾기

- 클러스터 내 SSE를 바탕으로 엘보우 방법이라고 하는 그래프를 사용하여 문제에 최적인 클러스터 개수 k 를 추정할 수 있음
- 직관적으로 생각했을 때 k 가 증가하면 왜곡은 줄어들 것
- 샘플이 할당된 센트로이드에 더 가까워지기 때문임
- 엘보우 방법 이면에 있는 아이디어는 감소 폭이 급격히 줄어드는 지점 (엘보우)의 k 값을 찾기
- k 값을 바꾸어 가며 왜곡 값을 그래프로 그리면 명확하게 알 수 있음



2) 엘보우 방법을 사용하여 최적의 클러스터 개수 찾기

```
distortions = []  
for i in range(1, 11):  
    km = KMeans(n_clusters=i,  
                init='k-means++',  
                n_init=10,  
                max_iter=300,  
                random_state=0)  
  
    km.fit(X)  
    distortions.append(km.inertia_)  
plt.plot(range(1, 11), distortions, marker='o')  
plt.xlabel('Number of clusters')  
plt.ylabel('Distortion')  
plt.tight_layout()  
plt.show()
```



03

평가 방식: 실루엣 계수



실루엣 계수로 군집 품질을 정량화

- 실루엣 분석은 클러스터 내 샘플들이 얼마나 조밀하게 모여 있는지를 측정하는 그래프 도구
 - 다른 군집 알고리즘에도 적용할 수 있음
- 실루엣 계수(silhouette coefficient)를 계산하려면 다음 세 가지 단계를 적용

① $a^{(i)}$: 클러스터 응집도 (cluster cohesion)

→ 같은 클러스터 내부 데이터들이 얼마나 가까이 모여 있는가

② $b^{(i)}$: 클러스터 분리도 (cluster separation)

→ 서로 다른 클러스터 간 거리가 얼마나 떨어져 있는가

③ 실루엣 계산

$$s^{(i)} = \frac{b^{(i)} - a^{(i)}}{\max \{b^{(i)}, a^{(i)}\}}$$



실루엣 계수의 의미

- 실루엣 계수는 -1과 1 사이 값을 가짐

$$s^{(i)} = \frac{b^{(i)} - a^{(i)}}{\max\{b^{(i)}, a^{(i)}\}}$$

클러스터 응집력 $a^{(i)}$
클러스터 분리도 $b^{(i)}$

- 1에 가까울수록 잘 군집됨 $\leftarrow b^{(i)} \gg a^{(i)}$
 - 0에 가까울수록 군집 경계에 위치 $\leftarrow b^{(i)} = a^{(i)}$
 - 음수면 잘못된 군집 배정
- why?
 - $b^{(i)}$ 는 샘플이 다른 클러스터와 얼마나 다른지 나타내고
 - $a^{(i)}$ 는 클러스터 내 다른 샘플과 얼마나 비슷한지 나타내기 때문임

좋은 클러스터링 : 내부는 뭉쳐있으면서 다른 클러스터와는 먼 상태
=> 응집도(거리)는 낮고 분리도는 높은 상태



사이킷런기반 실루엣 계수 계산

- 실루엣 계수는 “silhouette_score” 함수를 사용
- silhouette_score 함수는 모든 샘플에 걸쳐 평균 실루엣 계수를 계산

```
from sklearn.metrics import silhouette_score
kmp = KMeans(n_clusters=3,
             init='k-means++',
             n_init=10,
             max_iter=300,
             tol=1e-04,
             random_state=0)
kmp.fit(X) #학습
y_km = kmp.predict(X)#예측 클러스터

silhouette_score(X,y_km)
```

0.4999271626194841



사이킷런기반 실루엣 계수 계산

- silhouette_scores 값은 numpy.mean(silhouette_samples(...))처럼 계산하는 것과 동일함

```
from sklearn.metrics import silhouette_samples
silhouette_vals = silhouette_samples(X, y_km)
silhouette_avg = np.mean(silhouette_vals)

print(silhouette_vals)
print(silhouette_avg)
```

```
[0.6611825  0.58343915 0.33364576 0.27643761 0.41341881 0.53481666
0.6272443  0.14802218 0.6330197  0.55205447 0.47992824 0.02859872
0.64799481 0.6746723  0.38365138 0.62347356 0.58750186 0.64250055
0.21071219 0.28359516 0.02854402 0.50229717 0.61316491 0.41185683
0.59641819 0.59220937 0.38446454 0.45439527 0.03256845 0.72018433
0.61076411 0.40301651 0.37198505 0.57299739 0.11548126 0.74385562
0.25266335 0.45227805 0.54184849 0.46563913 0.67477154 0.60334216
0.60948171 0.7242502  0.71178505 0.73915975 0.56379625 0.51513487
0.57891348 0.64174145 0.55004398 0.55116427 0.55944442 0.45657692
0.36260507 0.65822616 0.36565777 0.69041135 0.42520283 0.60879084
0.42706445 0.45982541 0.59022122 0.53545897 0.0502389  0.50716627
0.05105976 0.57105587 0.62681344 0.33760271 0.29160277 0.60216194
0.4617408  0.23909497 0.71527091 0.5754084  0.3142701  0.22176925
0.10000147 0.7162794  0.56472038 0.52959256 0.45388271 0.52864707
0.62492275 0.71076915 0.09477712 0.73709794 0.57437473 0.58205795
0.72727087 0.55571485 0.74120478 0.43026018 0.6782505  0.62102864
0.59884674 0.26073867 0.56933646 0.51453571 0.4552654  0.73200232
0.60166864 0.65185849 0.63417531 0.62136127 0.6178633  0.40616298
0.44356407 0.27076386 0.58095782 0.66805062 0.61796079 0.57642315
0.55737113 0.58217215 0.14908536 0.22221153 0.57509457 0.61163626
0.60988827 0.31059676 0.69688982 0.50900285 0.59041931 0.61252731
0.57357502 0.55842089 0.59579091 0.70380984 0.56497962 0.67823886
0.4313649  0.74311356 0.20568262 0.61337808 0.48118088 0.63955965
0.49825498 0.65651851 0.09189172 0.56702694 0.60874829 0.69032874
0.60545539 0.12490107 0.16036033 0.7156249  0.17914066 0.12157595]
0.4999271626194841
```


04

실제 데이터로 사이킷런 실습



고객 분류/마케팅 데이터

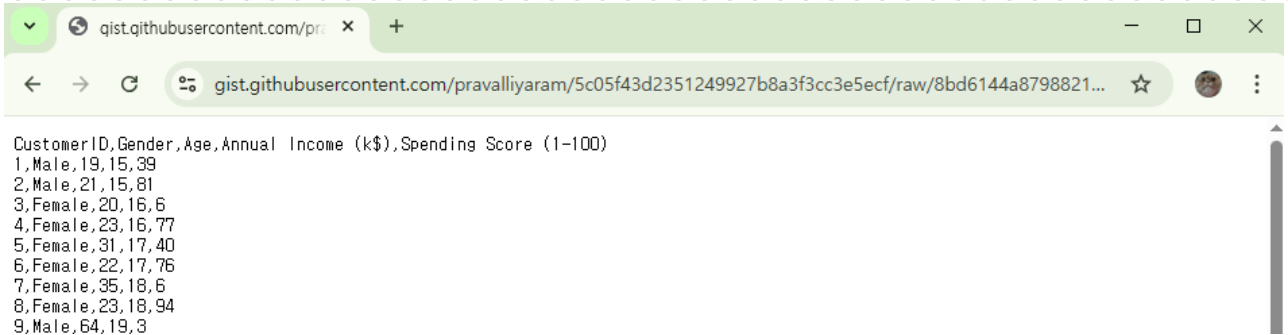
- 고객의 나이, 성별, 소득, 소비 점수를 기반으로 소비자 군집화나 타겟 마케팅 분석에 자주 사용
- 고객 세분화, 마케팅 전략 수립 등을 목표로 하는 경우가 많음
- “Mall_Customers.csv” 데이터 설명

열 이름	데이터 타입	설명
CustomerID	정수 (int)	고객 고유 ID
Gender	문자열 (str)	고객 성별 ("Male" 또는 "Female")
Age	정수 (int)	고객 나이
Annual Income (k\$)	정수 (int)	연 소득 (천 달러 단위)
Spending Score (1-100)	정수 (int)	고객의 소비 성향을 나타내는 점수 (1~100)



데이터 준비

- “Mall_Customers.csv”파일은 Github에 있음



#필요한 패키지 준비

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt
```

데이터 로딩

```
url = "https://gist.githubusercontent.com/pravalliyaram/5c05f43d2351249927b8a3f3cc3e5ecf/raw/8bd6144a87988213693754baaa13fb204933282d/Mall_Customers.csv"
df = pd.read_csv(url)
df.head(3)
```

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6



전처리

- 1) CustomerID열 제거
- 2) Gender열: category type 지정 후, 원 핫 인코딩
- 3) MinMax 정규화

#1)CustomerID열 제거

```
df.drop("CustomerID",axis=1,inplace=True)
```

#2)Gender열

```
df["Gender"]=df["Gender"].astype('category')#Gender열 category type 지정
```

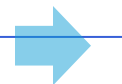
```
df = pd.get_dummies(df) #Gender열만 원 핫 인코딩
```

#3)MinMax 정규화

```
X = MinMaxScaler().fit_transform(df) #df에 적용
```

```
print(df.columns)
```

```
X[:5,]
```



```
Index(['Age', 'Annual Income (k$)', 'Spending Score (1-100)', 'Gender_Female',  
      'Gender_Male'],  
      dtype='object')
```

```
array([[0.01923077, 0.          , 0.3877551 , 0.          , 1.          ],  
       [0.05769231, 0.          , 0.81632653, 0.          , 1.          ],  
       [0.03846154, 0.00819672, 0.05102041, 1.          , 0.          ],  
       [0.09615385, 0.00819672, 0.7755102 , 1.          , 0.          ],  
       [0.25        , 0.01639344, 0.39795918, 1.          , 0.          ]])
```



최적의 k값 찾기

- 본격적으로 학습 전 엘보우 기법으로 최적의 클러스터 개수 정하기

```
inertias = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X)
    inertias.append(km.inertia_)
```

#그래프

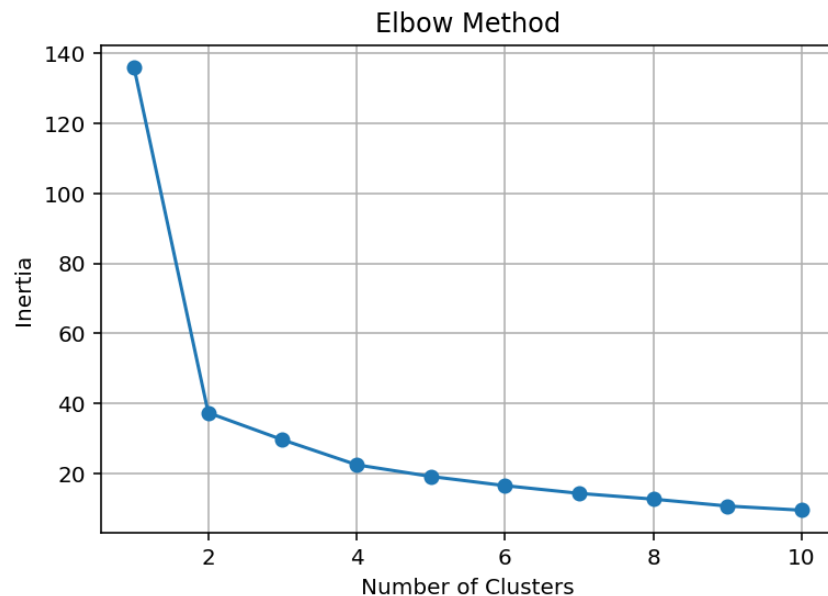
```
plt.plot(range(1, 11), inertias, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
plt.ylabel("Inertia")
plt.grid()
plt.show()
```

#차이가 큰 인덱스

```
arr = np.array(inertias)
diffs = np.diff(arr) #원소간의 차
diffs = np.abs(diffs) #절대값
n=np.argmax(diffs)+2
```

```
print(diffs)
```

```
print(f"최적의 k 개수: {n}")
```



n	1	2	3	4	5	6	7	8	9	10
		+1								
inertias	0	1	2	3	4	5	6	7	8	9
IDX										
value	135.96	37.27	29.55	22.39	19.06	16.43	14.22	12.57	10.6	9.4
diffs	0	1	2	3	4	5	6	7	8	
IDX		+1								
value	98.69	7.72	7.16	3.33	2.21	1.65	1.97	1.97	1.2	



학습하기

■ K-means 학습

```
# KMeans
kmeans = KMeans(n_clusters=n,
                 init='k-means++',
                 n_init=10,
                 max_iter=300,
                 tol=1e-04,
                 random_state=0)
labels = kmeans.fit_predict(X)
print(labels)
```



```
[0 0 1 1 1 1 1 1 0 1 0 1 1 1 0 0 1 0 0 1 0 0 1 0 1 0 1 0 1 0 1 1 0 1 0 0 1 1 1
 1 1 1 1 0 0 1 1 1 1 1 1 1 1 0 1 0 1 0 1 0 1 0 0 0 1 1 0 0 1 1 0 1 0 1 1 1
 0 0 1 0 1 1 0 0 0 1 1 0 1 1 1 1 1 0 0 1 1 0 1 1 0 0 1 1 0 0 0 1 1 0 0 0 0
 1 1 0 1 1 1 1 1 0 1 1 0 1 1 0 0 0 0 0 0 1 1 0 1 1 0 0 1 1 0 1 1 0 0 0 1
 1 0 0 0 1 1 1 1 0 1 0 1 1 1 0 1 0 1 1 0 0 0 0 0 1 1 0 0 0 0 1 1 0 1 1
 0 1 0 1 1 1 1 0 1 1 1 1 0 0 0]
```

■ 센트로이드 좌표: 클러스터 숫자 별로 각 열에 대한 센트로이드 값

```
pd.DataFrame(kmeans.cluster_centers_, columns = df.columns)
```

	Age	Annual Income (k\$)	Spending Score (1-100)	Gender_Female	Gender_Male
0	0.419362	0.387109	0.484810	-4.440892e-16	1.000000e+00
1	0.386504	0.362705	0.515579	1.000000e+00	4.440892e-16



학습 결과

■ 실루엣 평가

```
silhouette_score(X, labels)
```



0.6346497426496239

■ 결과 분석

```
result = df.copy()  
result["cluster"] = labels  
result.head()
```

	Age	Annual Income (k\$)	Spending Score (1-100)	Gender_Female	Gender_Male	cluster
0	19	15	39	False	True	0
1	21	15	81	False	True	0
2	20	16	6	True	False	1
3	23	16	77	True	False	1
4	31	17	40	True	False	1

```
temp = (result["Gender_Female"] == result["cluster"])  
np.mean(temp)
```

1.0

완벽히 일치

- 클러스터링 결과가 Gender 기준으로 명확히 나뉘는 경우, 성별이 군집을 나누는 중요한 변수임을 시사
- 실루엣 점수가 0.635로 높게 나온 이유도 성별이 뚜렷한 클러스터 경계를 제공



SUMMARY

- K-평균 : 데이터를 비슷한 특성을 가진 여러 그룹으로 나누는 대표적인 군집화 방법
 - ① 각 클러스터의 중심인 센트로이드를 기준으로 가까운 데이터끼리 묶기
 - ② 중심을 반복적으로 갱신함
- 한계점
 - 사전에 클러스터 개수 k 를 직접 정해야함 -> 엘보우 기법으로 적절한 k 선택
 - 초기 중심값에 따라 결과가 달라질 수 있음 -> k-means++로 개선
- 평가 방법 : 실루엣 계수(범위 $-1 \sim 1$)를 사용하여 군집 품질을 평가할 수 있음
 - 1에 가까울수록 : 같은 클러스터 내부는 잘 뭉쳐 있고 다른 클러스터와는 잘 분리된 상태
 - 0에 가까울수록 : 군집 경계에 위치하여 어느 클러스터에 속하는지 애매한 상태
 - 음수일수록 : 다른 클러스터에 더 가까워 잘못 군집되었을 가능성이 큰 상태
- 실습 흐름
 - 데이터 전처리 → K-평균 군집화 → 엘보우 방법과 실루엣 계수로 평가 → 군집별 패턴 해석 및 분석



Q&A

궁금한 사항은 아래 방법으로 문의 바랍니다.



이메일
dana1112@bible.ac.kr



LMS 쪽지

THANK YOU FOR YOUR ATTENTION!

한국성서대학교 | 인공지능융합학부 | 머신러닝